AIG 130 CLOUD COMPUTING FOR MACHINE LEARNING

LAB #05

GROUP REPORT

Automated Machine Learning (AutoML) with AWS AutoGluon

Group #04
Srimedha Jaggavarapu

Alexandre Papouchine

AnuSri Saini

Chao Chen

CONTENTS

1. INTRODUCTION

This report documents our exploration of Automated Machine Learning (AutoML) platforms as part of Lab Assignment. AutoML refers to the process of automating the end-to-end application of machine learning to real-world problems. It covers everything from raw data preprocessing and feature engineering to model selection, hyperparameter optimization, and ensemble construction.

For this lab, we selected AWS AutoGluon as our AutoML platform. We applied it to the classic Titanic survival prediction dataset, a well-known problem from Kaggle.

2. AUTOML PLATFORMS

Google AutoML, which is part of Vertex AI, is a set of tools that lets you build machine learning models using a simple "point-and-click" website instead of writing code. It is especially powerful for analyzing "unstructured" data, such as identifying objects in images, understanding videos, and processing human language. While it uses Google's advanced technology to automatically find the best model for your specific project, it is a proprietary service that only works on the Google Cloud Platform. This means that while it is very user-friendly and includes helpful tools for labeling data, the costs can add up quickly, and you cannot run the software on your own computer or other cloud services.

AutoGluon is an open-source Automated Machine Learning (AutoML) library developed by AWS that simplifies the creation of high-accuracy models across tabular, text, image, and time-series data with minimal Python code. It is designed to automate the entire machine learning pipeline, including data preprocessing, feature engineering, and model selection across diverse algorithm families like neural networks and tree-based models. A standout feature of the library is its use of advanced ensemble techniques, such as multi-layer stacking and weighted ensembles, which automatically combine multiple models to maximize predictive performance. In a laboratory setting, AutoGluon is particularly advantageous because it integrates seamlessly with Google Colab without requiring external cloud infrastructure, and its built-in leaderboard allows users to easily rank and compare model performance, making it an efficient "out-of-the-box" solution for structured datasets.

Azure AutoML is Microsoft's professional tool for building AI models, and it is designed to be used by both beginners and experts. We can either use a simple website interface or write code to build your models, making it very flexible. Its biggest strength is its focus on safety and rules, providing special tools to check if a model is fair, unbiased, and easy to explain which is very important for industries like healthcare or banking. It also works perfectly with other Microsoft products like Power BI and Office 365, making it a great choice for companies that already use those programs. Additionally, it is excellent at predicting future trends, like sales or weather, and includes advanced features to track every version of a model as it is being built.

Dimension	Google AutoML	AWS AutoGluon	Azure AutoML
Interface	GUI(no code)	Python APT only	GUI + Python SDK
Open-source	No(proprietary)	Yes(Apache 2.0)	No(proprietary)
Cloud lock-in	GCP only	Cloud-agnostic	Azure only
Best data type	Images, text, video	Tabular, multimodal	Tabular, time series
Ensemble method	NAS + Transfer learning	Multi-layer stacking	Voting + stacking
Pricing model	Pay-per-node-hour	Free (compute only)	Pay-per-experiment
MLOPs support	Via Vertex AI	Manual/SageMaker	Built-in (AzureML)
Compilance Tools	Limited	None built-in	Responsible AI suite

3. DATASET: TITANIC SURVIVAL PREDICTION

We used the Titanic dataset from the Kaggle competition (kaggle.com/c/titanic). The dataset consists of two files:

- train.csv – 891 passenger records with the target label Survived (0 = did not survive, 1 = survived)
- test.csv – 418 passenger records without the Survived label, used for prediction

4. IMPLEMENTATION

We used Google Colab for this project because it provides free computing power and makes it easy to upload our data files. After uploading the training and testing sets, we used AutoGluon's specialized format to prepare the data. The training was simple to start with one command, where we told the program to predict who "Survived" based on accuracy. We set a two-minute time limit and used the "best quality" setting to ensure the model was as precise as possible. AutoGluon did the heavy lifting by automatically organizing the data, testing many models at once, and combining the best ones. Finally, we checked the leaderboard to see which models performed best and used the top version to make our final predictions.

4.1 AUTOGLUON PLATFORM IMPLEMENTATION

In this lab, we learned about several powerful things AutoGluon can do:

- **Automatic Prep Work:** The tool fixed missing data and organized different types of information all by itself, without us needing to do it manually.
- **Fast Multi-Tasking:** It trained many different types of models at the same time and finished everything within our two-minute limit.

- **Smart Combining:** It used techniques called "bagging" and "stacking" to build more reliable models and then combined the best ones into a "super-model" for better results.
- **Clear Ranking:** The leaderboard showed us exactly how each model performed, making it easy to see which ones were the most accurate.

The most impressive part is that we did all of this from loading the raw files to finishing the project with fewer than 20 lines of code. This makes machine learning much easier for everyone to use.

However, there are a few things to keep in mind. Because we only gave it two minutes, the tool couldn't try every possible setting; more time would likely lead to better accuracy. Also, because the tool uses a bit of randomness to find the best settings, your results might change slightly every time you run it. Finally, while the tool is very effective, it can sometimes be hard to see exactly why it made certain technical choices behind the scenes.

5. RESULTS

The following table shows the models AutoGluon trained and their test/validation accuracy scores. AutoGluon explored multiple model families including LightGBM (gradient boosting), and combined the best into weighted ensemble models.

	model	score_test	score_val	eval_metric	pred_time_test	pred_time_val	fit_time	pred_time_test_marginal	pred_time_val_marginal	fit_time_marginal	stack_level	can_infer	fit_order
0	RandomForestEntr_BAG_L1	1.000000	0.815937	accuracy	0.145226	0.162350	0.977323	0.145226	0.162350	0.977323	1	True	4
1	RandomForestGini_BAG_L1	1.000000	0.822671	accuracy	0.149132	0.227013	1.335358	0.149132	0.227013	1.335358	1	True	3
2	ExtraTreesEntr_BAG_L1	1.000000	0.819304	accuracy	0.157962	0.158963	0.810277	0.157962	0.158963	0.810277	1	True	7
3	ExtraTreesGini_BAG_L1	1.000000	0.812570	accuracy	0.176745	0.159225	1.088118	0.176745	0.159225	1.088118	1	True	6
4	WeightedEnsemble_L3	0.913580	0.850730	accuracy	0.829962	0.798416	147.949420	0.004067	0.001216	0.104803	3	True	11
5	CatBoost_BAG_L1	0.912458	0.848485	accuracy	0.063079	0.035538	34.699786	0.063079	0.035538	34.699786	1	True	5
6	WeightedEnsemble_L2	0.912458	0.849607	accuracy	0.657446	0.586723	94.804705	0.004032	0.001286	0.117693	2	True	9
7	LightGBMXT_BAG_L2	0.909091	0.847363	accuracy	0.825896	0.797200	147.844618	0.066012	0.063473	25.418724	2	True	10
8	LightGBM_BAG_L1	0.897868	0.842873	accuracy	0.106469	0.148290	27.738881	0.106469	0.148290	27.738881	1	True	2
9	NeuralNetFastAI_BAG_L1	0.850730	0.846240	accuracy	0.283242	0.163924	57.841592	0.283242	0.163924	57.841592	1	True	8
10	LightGBMXT_BAG_L1	0.849607	0.837262	accuracy	0.169215	0.051537	26.955207	0.169215	0.051537	26.955207	1	True	1

Our best model was a specific version of LightGBM, which reached an accuracy of about 84.96%. Interestingly, the combined "ensemble" models got the same score, meaning the single best model was already as good as it could get for this specific run.

One thing we noticed is that if you run the same code again, the results change slightly each time. This happens because the tool uses a bit of randomness when searching for the best settings. Finally, we successfully predicted results for all 418 passengers in our test file. When we checked the first few rows of our results, we could see clear "0" or "1" predictions showing whether each person was predicted to survive.

6. CONCLUSION

This lab gave us hands-on practice with AutoGluon, a powerful tool from AWS that does a lot of the hard work in machine learning for you. By using it to predict who survived the Titanic, we showed that the tool can get great results—around 85% accuracy—with very little code and almost no manual setup.

Tools like this are great because they allow anyone to build and test models very quickly. However, they don't replace humans entirely. You still need to use your own judgment to decide which information is important to include and how to measure success.

Overall, this exercise showed that AutoGluon is a fantastic starting point for solving data problems. It proves that modern tools can now do in minutes what used to take hours or days of manual work.

7. REFERENCES

- Amazon Web Services. AutoGluon Documentation. https://auto.gluon.ai/
- Kaggle. Titanic – Machine Learning from Disaster. https://www.kaggle.com/c/titanic
- Erickson, N. et al. (2020). AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. arXiv:2003.06505.
- Google Colab. https://colab.research.google.com